

Phase 1.1

Wednesday, 20 May 2026 9:53 AM

PHASE-1

Core Local Foundation
(Already Done)

You built:

- FastAPI
- APIs
- prompts
- stories
- activities
- uploads
- reflections
- mock/basic integrations



PHASE-1.1

REAL INTEGRATIONS + STABILIZATION
(NOW)

Goal:

Take everything from Phase-1 and make it:

- real
- stable
- production-ready
- observable

WITHOUT adding new business intelligence.

This is VERY important.



PHASE-1.1 SCOPE

ONLY improve:

- real integrations
- infra quality
- safety
- deployment
- observability
- reliability

NO:

- RAG
- recommendation engine
- personalization
- learning graph
- vector DB



PHASE-1.1 MODULES



STEP-16 — Real OpenRouter Integration

Replace mock provider fully.

Add

- Gemini Flash
- DeepSeek
- Qwen fallback

Validate

- real responses
- timeout handling
- retry handling



STEP-17 — Structured AI Output Enforcement

Currently:

AI may return random formats.

Fix that.

Enforce JSON Responses

Example:

```
{  
  "title": "",  
  "story": "",  
}
```

```
"moral": "",  
"activity_hint": ""  
}
```

Why

Your frontend stability depends on this.

STEP-18 — AI Safety & Validation Layer

Critical for child platform.

Validate

- profanity
- unsafe content
- extremely long outputs
- malformed JSON

Add

- fallback response
- validation middleware

STEP-19 — Prompt Stabilization

Right now prompts are unstable.

Improve

- system prompts
- child tone consistency
- output consistency
- multilingual readiness

Add

Prompt metadata:

- version
- category
- active status

ONLY local metadata.

No DB needed yet.

STEP-20 — Real Image Reflection Integration

Currently probably partial.

Now make real.

Integrate

Vision-capable model.

Flow

image upload
→ vision model
→ reflection generation

Ensure

Always:

- positive
- encouraging
- curiosity-driven

STEP-21 — Production Logging

Now improve observability.

Add

- request logs
- AI logs
- error logs
- latency logs

Very Important

Track:

model
tokens
latency
failures

● STEP-22 — Analytics Provider Integration

Now connect real analytics.

Use

[PostHog](#)

Track

- story generation
- activity generation
- uploads
- reflections
- failures

Goal

Understand:

What are users actually using?

● STEP-23 — Error Monitoring

Now add production monitoring.

Use

[Sentry](#)

Monitor

- API failures
- AI failures
- upload failures
- crashes

● STEP-24 — Background Processing

Prevent blocking APIs.

Use

- Redis
- Dramatiq

Move Async

- reflections
- long story generations
- analytics events

● STEP-25 — File Storage Upgrade

Move away from local storage.

Use

[Cloudflare R2](#)

OR

[Supabase Storage](#)

Recommended:

Supabase Storage initially.
Simpler.

● STEP-26 — API Security Foundation

Before frontend/mobile.

Add

- CORS
- request validation
- rate limiting
- upload size limits

● STEP-27 — Deployment Setup

Now make backend publicly accessible.

Setup

Setup

- Docker
- Railway deployment
- env management

Use:

[Railway](#)

● STEP-28 — CI/CD Foundation

Now automate deployments.

Add

GitHub Actions:

- lint
- test
- deploy

● STEP-29 — API Documentation Cleanup

Now polish APIs.

Improve

- Swagger descriptions
- response schemas
- examples
- error responses

● STEP-30 — Phase-1.1 Stabilization

Final cleanup.

Validate Entire System

- AI stable
- uploads stable
- analytics working
- deployment stable
- monitoring working



IMPORTANT UNDERSTANDING

After Phase-1.1:

you now have:

REAL production-ready AI backend

NOT:

prototype backend

Huge difference.



ONLY AFTER THIS

Move to:

Phase-2

(Intelligence + Personalization)

Because NOW:

- real usage exists
- real AI outputs exist
- analytics exist
- monitoring exists
- production infra exists